Replication of

# RANDOMNESS IN CARD SHUFFLING

# Introduction

This replication project investigates how randomness emerges in card shuffling through numerical simulation, inspired by classic results on the mathematics of shuffling, including studies on how many shuffles are needed to randomize a deck. It involves reviewing relevant research, implementing standard shuffling models (such as riffle, overhand, and pile shuffles), and replicating key theoretical results computationally. By analyzing how simulated shuffles converge to randomness under different metrics and assumptions, the project aims to build intuition for probabilistic mixing, evaluate the gap between theory and practice, and highlight the role of simulation in understanding stochastic processes.

# Project Goals

Read and understand key papers on card shuffling

Implement mathematical models for different shuffle ways by python

Exploring **Randomness** in card shuffling through numerical simulation

# KEY QUESTION

How many shuffles to **randomize** a deck of cards?

**OR**

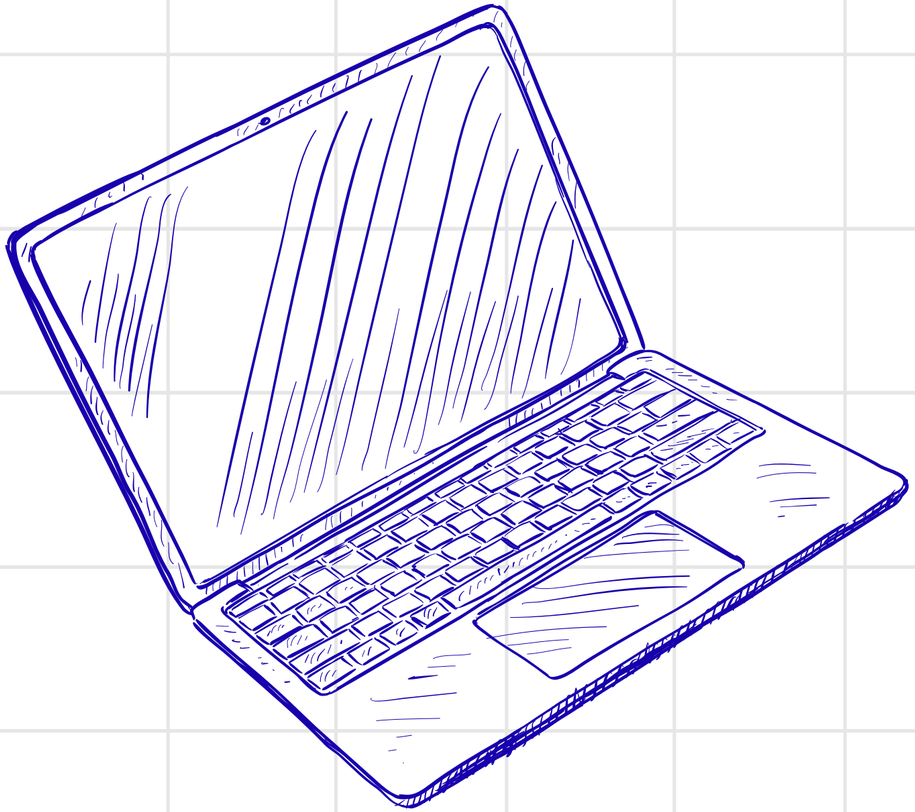
How many shuffles are needed to make a deck of cards "**random enough**"?

# IDEA BEHIND SHUFFLE

## Markov Chain & Mixing Time

**Markov chain** as a card shuffling chain is a stochastic process where the next state depends only on the current state.

**Mixing Time** is a measure of how many steps are needed until the card position distribution is close to uniform



# BASIC ASSUMPTION & DEFINITION

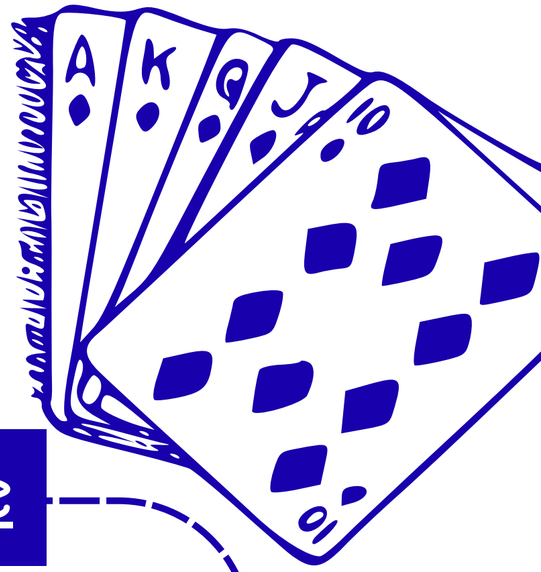
The default deck we shuffle is in its initial order and have 52 cards (we use only 4 suits and ignore jokers) if we get n card, we label them as from 0 to n-1

Randomness means each card has an equal probability of appearing in each position.

A deck is random if its distribution is close to the uniform distribution over all permutations.

Measured by Metrics

# Shuffle Ways



## riffle

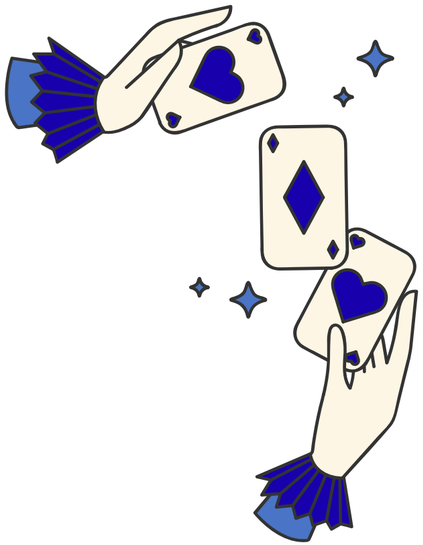
A standard shuffling technique in which half of the deck is held in each hand with the thumbs inward, then cards are released by the thumbs so that they fall to the table interleaved.

## overhand

The shuffling technique where someone gradually transfer the deck from one hand to another hand by sliding off small packets from the top of the deck with their thumb

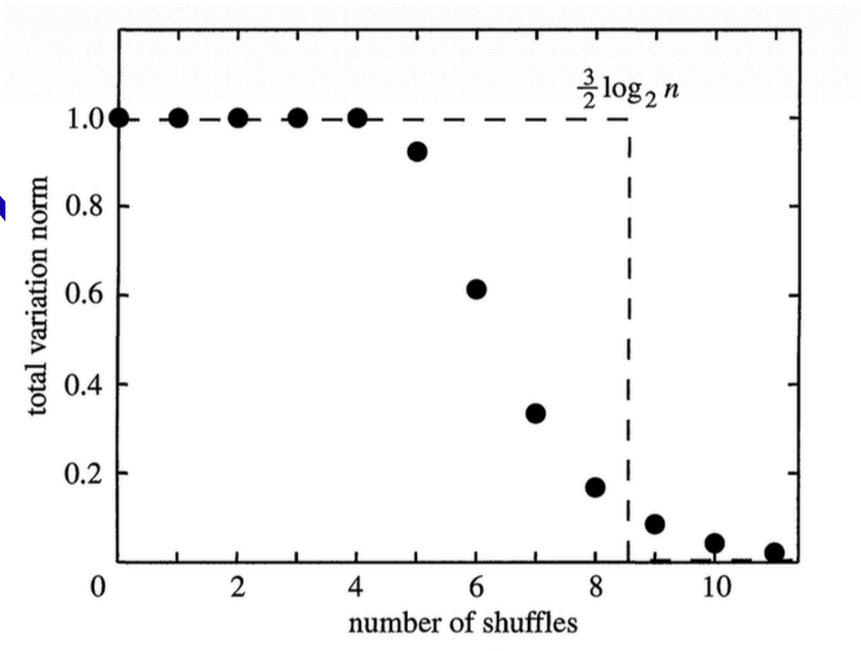
## Pile

A shuffling technique in which the deck is distributed into several piles by placing one card at a time into each pile in sequence, then the piles are gathered together to reform the deck.



# RIFFLE SHUFFLE

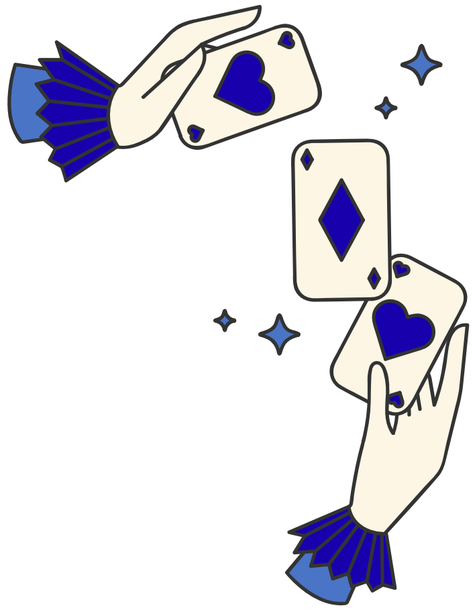
Simulated with **Gilbert-Shannon-Reeds(GSR)** model. In our simulation, we cut the deck into two packets, use Binomial to decide the cut point, and combined with the probability cut_p for each card goes to “left hand”. After cutting, the two packets are then interleaved probabilistically, we choose cards one by one from left or right hand with probability proportional to the number of remaining cards in each packet.




A **cut-off phenomenon** happens under total variation distance as measurment of randomness. It says “ Through step k = 4, virtually no reduction is achieved, and the total variation distancedoes not fall below 0.5 until step k = 7.” This is the origin of 'it takes seven shuffles to randomize a deck of cards'. (L. N & L. M)

$\sim \frac{3}{2} \log_2 n$ shuffles to random for total variation distance

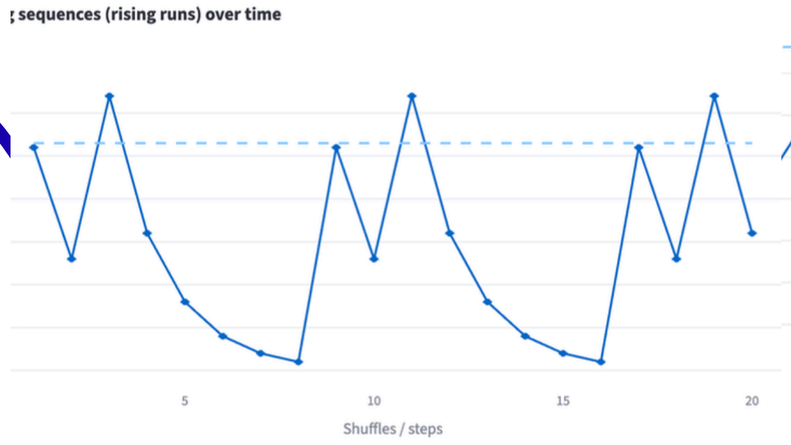
$\sim \log_2 n$ shuffles to random for (info) entropy

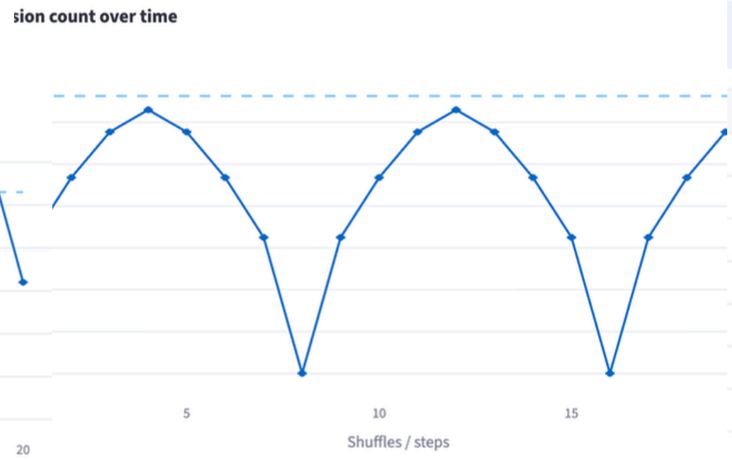


# PERFECT RIFFLE

A perfect riffle shuffle is a completely **deterministic** shuffle method, not a random process.
It involves first precisely dividing the deck into two halves. If there are 2n cards, this is divided into an upper and lower set of n cards each. Then, these two halves are strictly alternated, with one card coming from one half and one from the other.

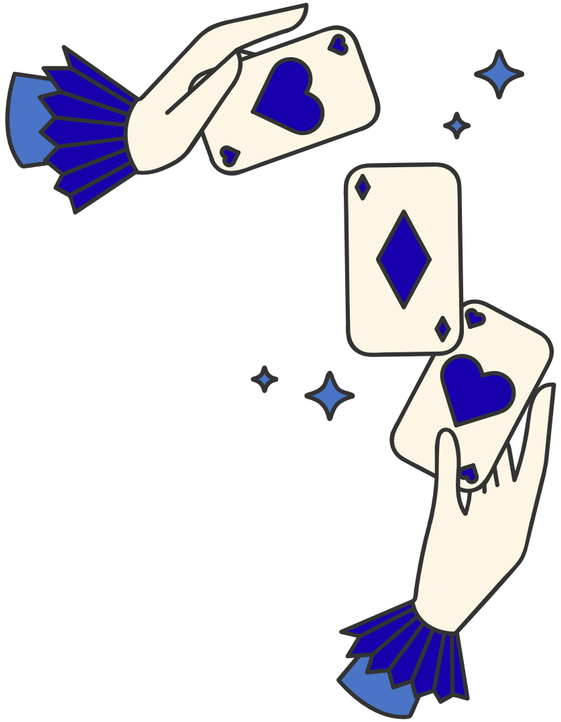
The shuffle process happens **periodic**, and **back to the deck's origianl order** after certain amount of shuffle based on the deck size



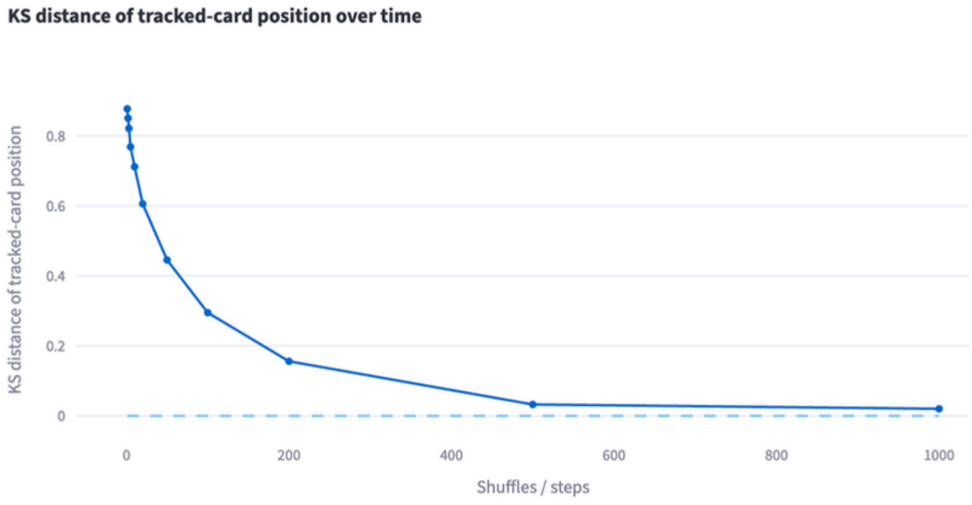

here returned_to_original shows at which step the deck order is back to its original order

| | steps | runs | inversions | returned_to_original |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | ☑ |
| 1 | 1 | 26 | 325 | ☐ |
| 2 | 2 | 13 | 468 | ☐ |
| 3 | 3 | 32 | 577 | ☐ |
| 4 | 4 | 16 | 630 | ☐ |
| 5 | 5 | 8 | 577 | ☐ |
| 6 | 6 | 4 | 468 | ☐ |
| 7 | 7 | 2 | 325 | ☐ |
| 8 | 8 | 1 | 0 | ☑ |
| 9 | 9 | 26 | 325 | ☐ |



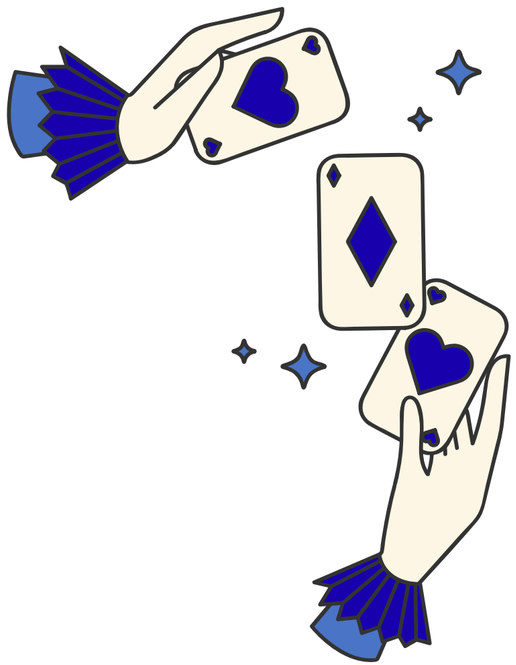
# OVERHAND SHUFFLE

Simulated with **Pemantle style model**. In our simulation, if there are n cards, between each adjacent card there are (n-1) possible cut point, each independently with the same probability $p \in (0, 1)$, so we get the number of cut points is a Binomial(n-1,p) random variable with larger p means stronger mixing. Then the deck split into packets according to these cutpoints with each packets remain in the same order, but they are reversed internally




In essay we know that previously "pemantle introduced and analyzed a probabilistic model for the overhand shuffle and established upper and lower bounds of order $n^2 \log n$ and $n^2$, respectively, for the mixing time on a deck of n cards. "
Then "establish a lower boud of order $n^2 \log n$ use Wilson tech" (Johan J)

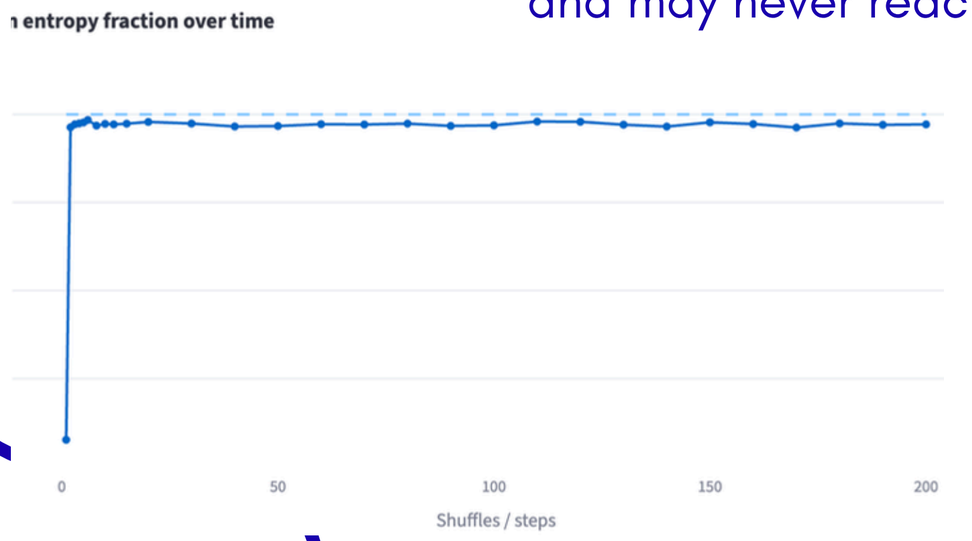
$\Theta(n^2 \log n)$ shuffles to random for total variation distance

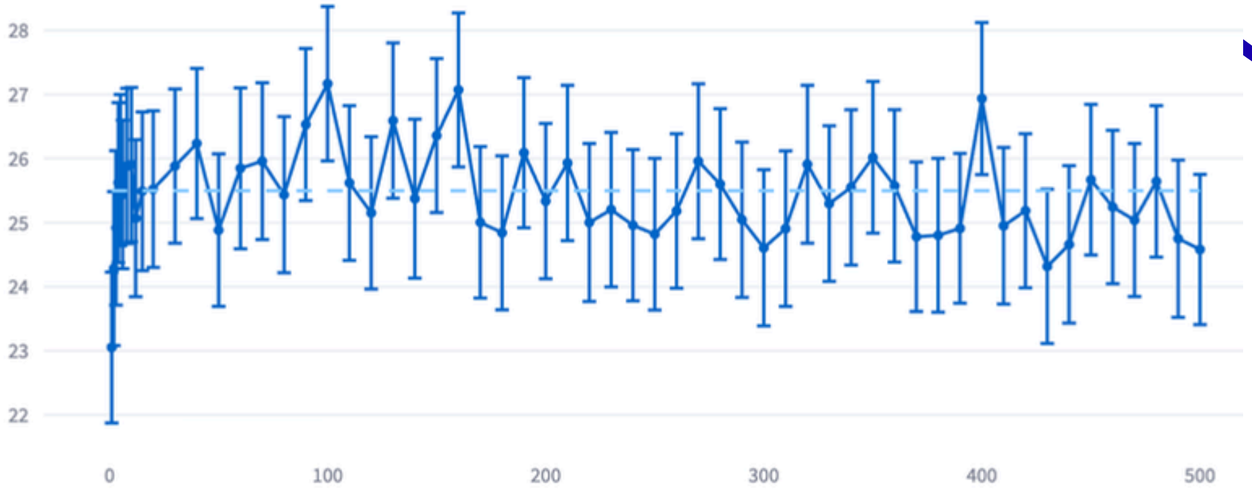


# PILE SHUFFLE

Simulated with **k-Pile Dealing Model**. In our simulation, this model separates the deck into k piles, and then collected to form a new deck.
If with random_pickup = True, order of the piles is randomly shuffled before collecting them.
The relative order of cards within each pile is preserved, but the overall structure of the deck changes depending on the number of piles and the order in which they are picked up.

**Limit:** It behaves like a permutation with structure (even cycles) and may never reach uniform distribution



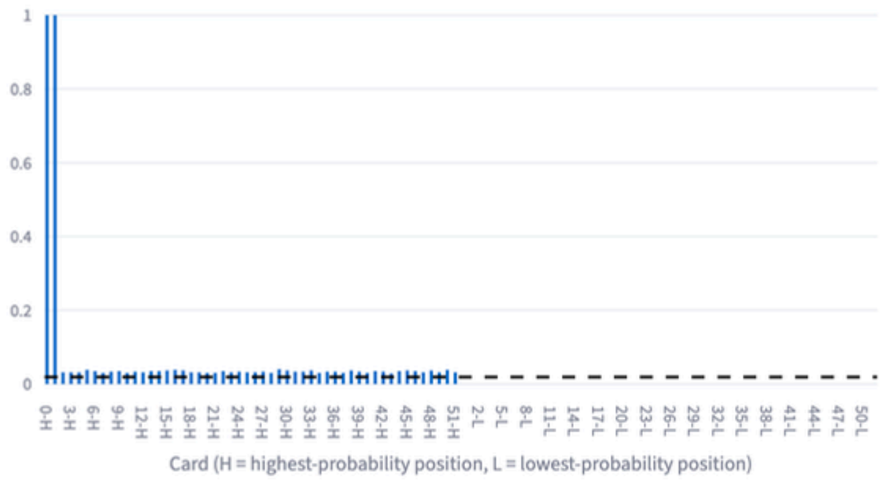
# CHEATER

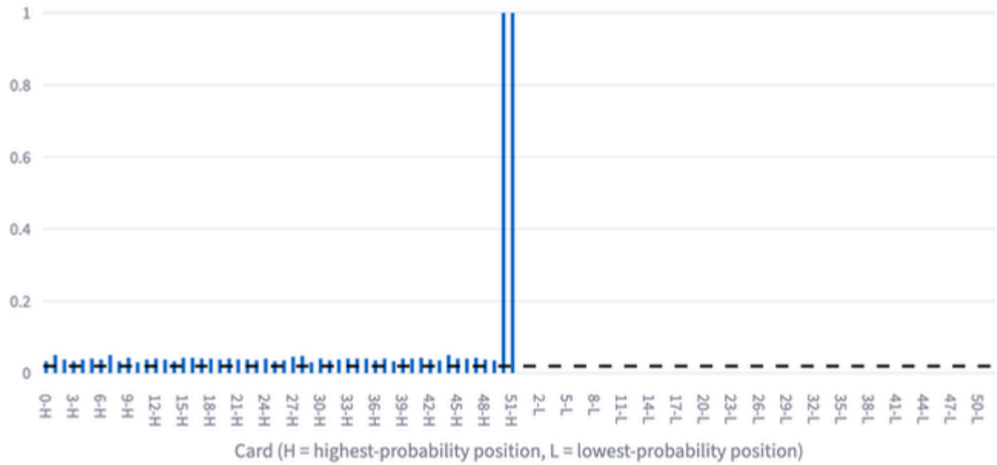
## CHEAT MODE

**keep_top:** Restores the original top k cards back to the top, the top portion of the deck is never truly mixed.

**keep_bottom:** Restores the original bottom k cards back to the bottom, the bottom portion behaves like an anchored block.

**return_tracked_to_top:** Take the tracked cards from their current position and places it on top after every shuffle, the tracked card never spreads out.



# Metrics

These are the metrics used to measure the randomness of the deck card. Different metrics capture different aspects of randomness, so using multiple metrics provides a more complete evaluation.

KS Distance

TV Distance

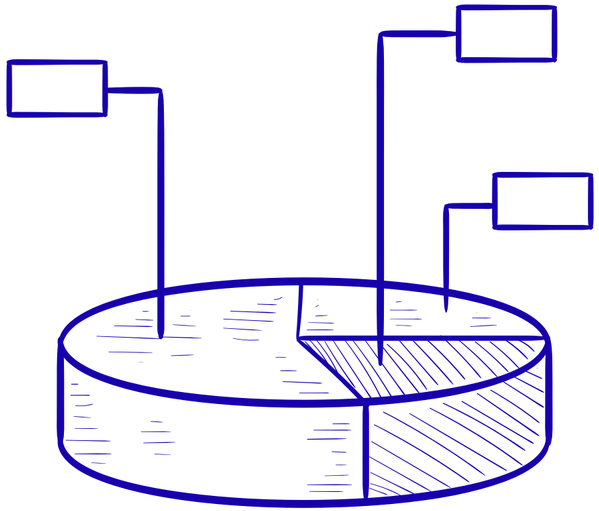
Position Entropy

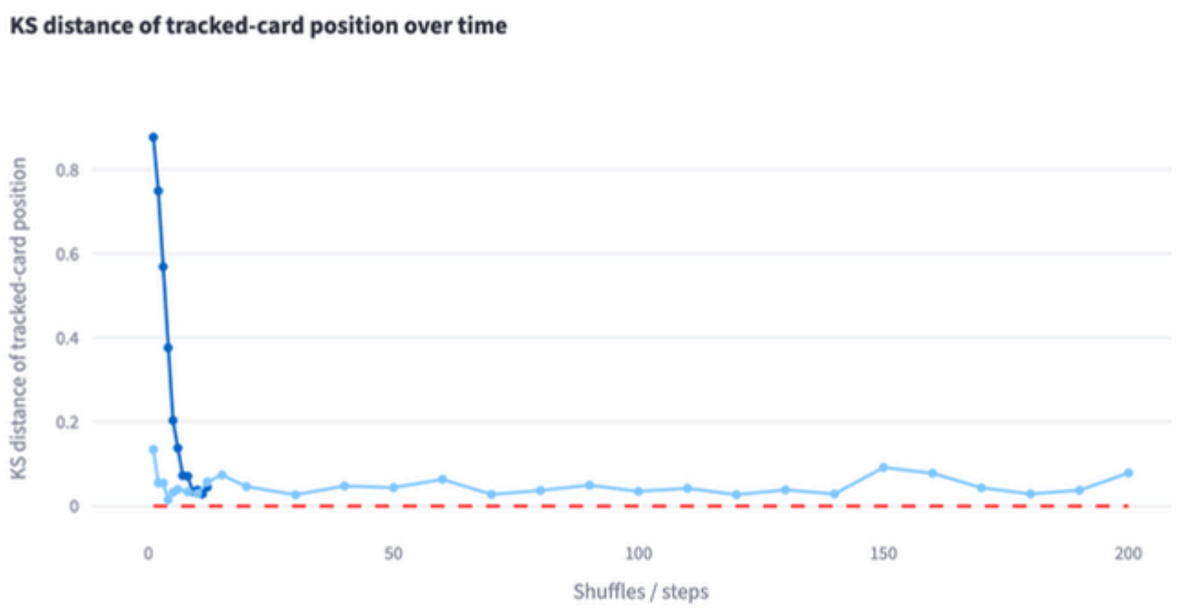
Inversion Count

Rising Runs

Mean Position

# KOLMOGOROV–SMIRNOV(KS) DISTANCE

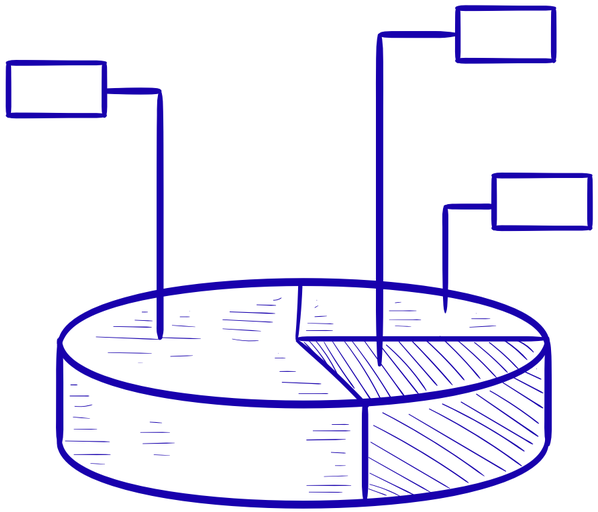


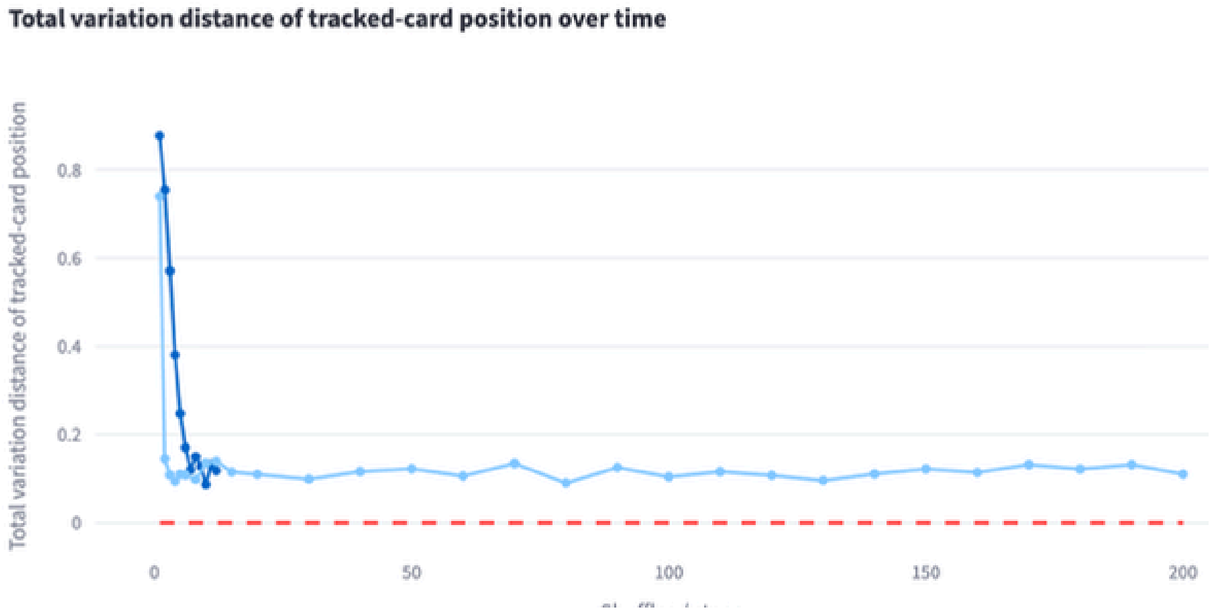



Compute the KS distance(largest) between the empirical distribution of tracked-card positions and the uniform distribution.
This measures how different the observation of card-position distribution is from a perfectly uniform one

If the shuffle is perfect, every position should have the probability: $\frac{1}{n}$

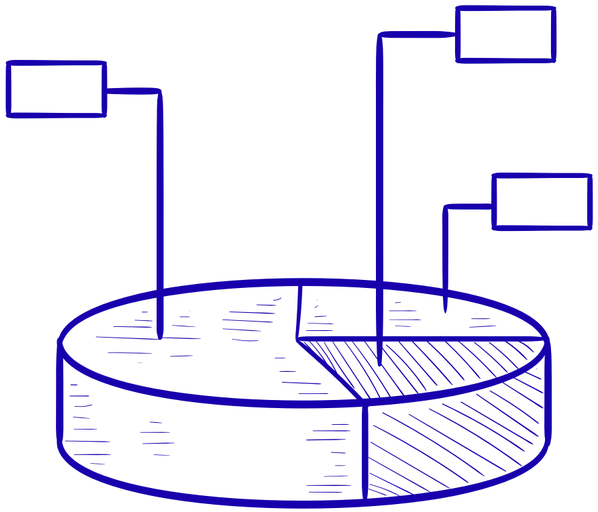
# TOTAL VARIATION(TV) DISTANCE

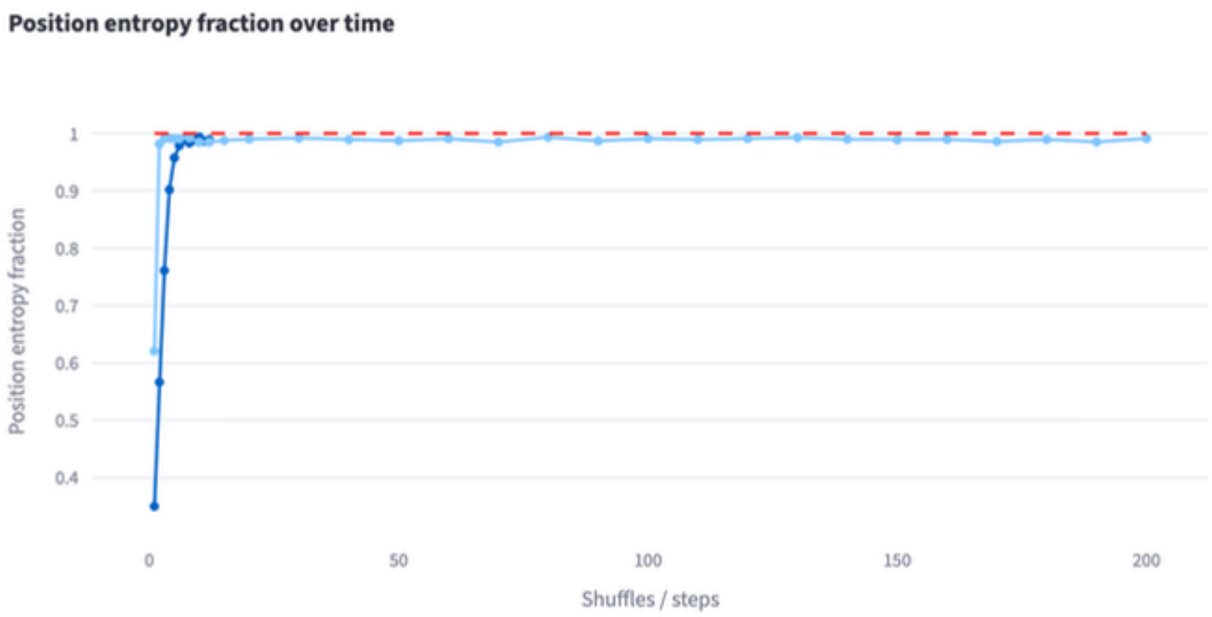





Compute the TV distance(global total) between the empirical distribution of tracked-card positions and the uniform distribution.
This measures how different the observation of card-position distribution is from a perfectly uniform one

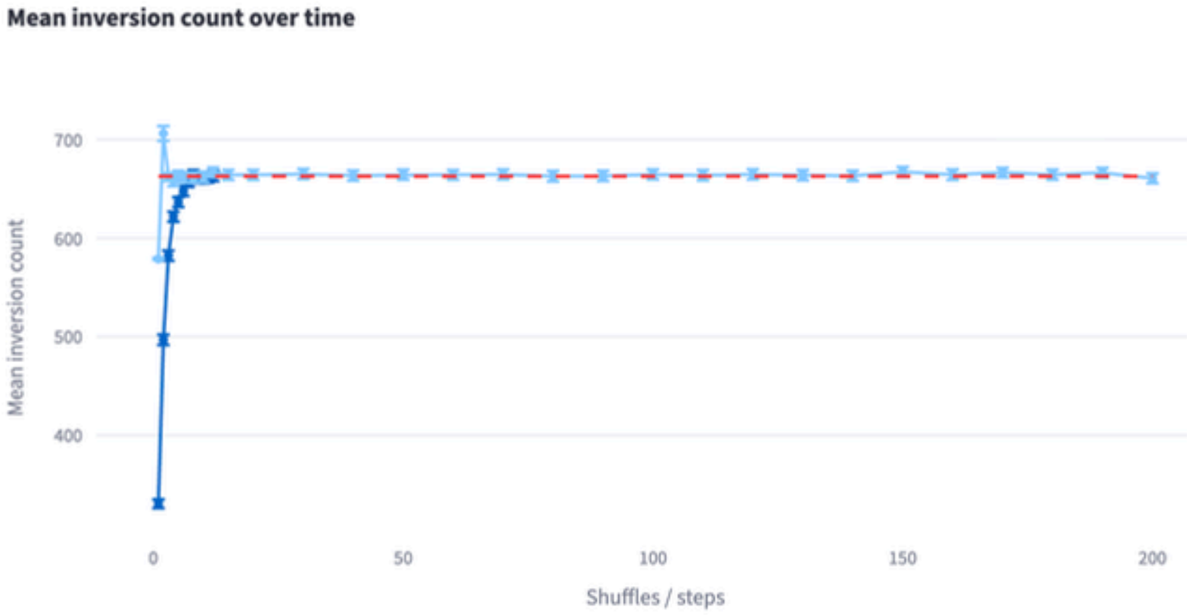
# POSITION ENTROPY

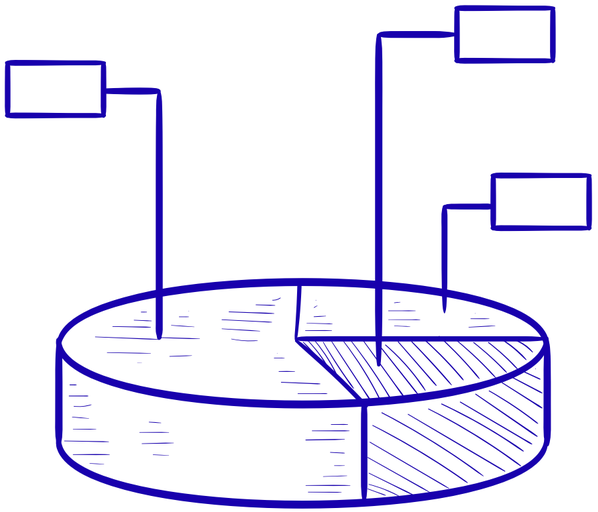





Evaluate how evenly the tracked card is distributed, which quantifies uncertainty in the distribution.
This measures how spread out the tracked card's position probabilities are, indicating how evenly it is distributed across positions.

# INVERSION COUNT

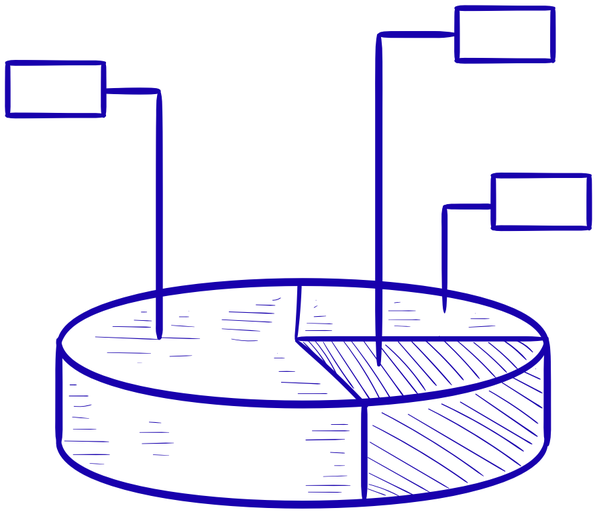


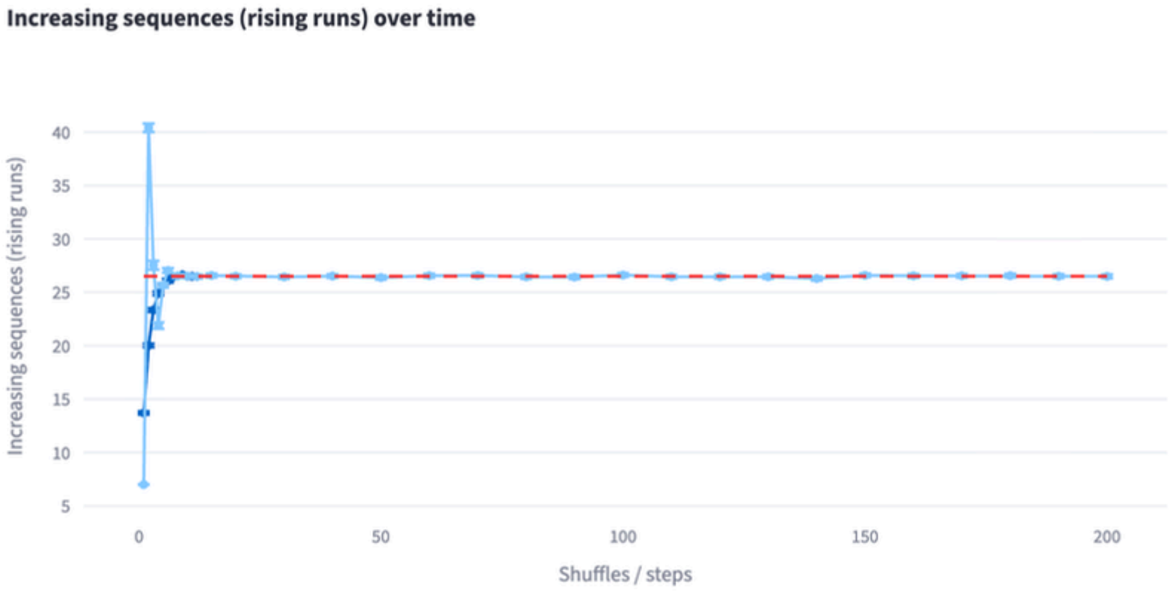



Measure global disorder of the entire deck, which answer “how far is the deck from its original sorted order”.

This counts how many pairs of cards are disorder, which answer how far is the deck from its original sorted order

# RISING RUNS

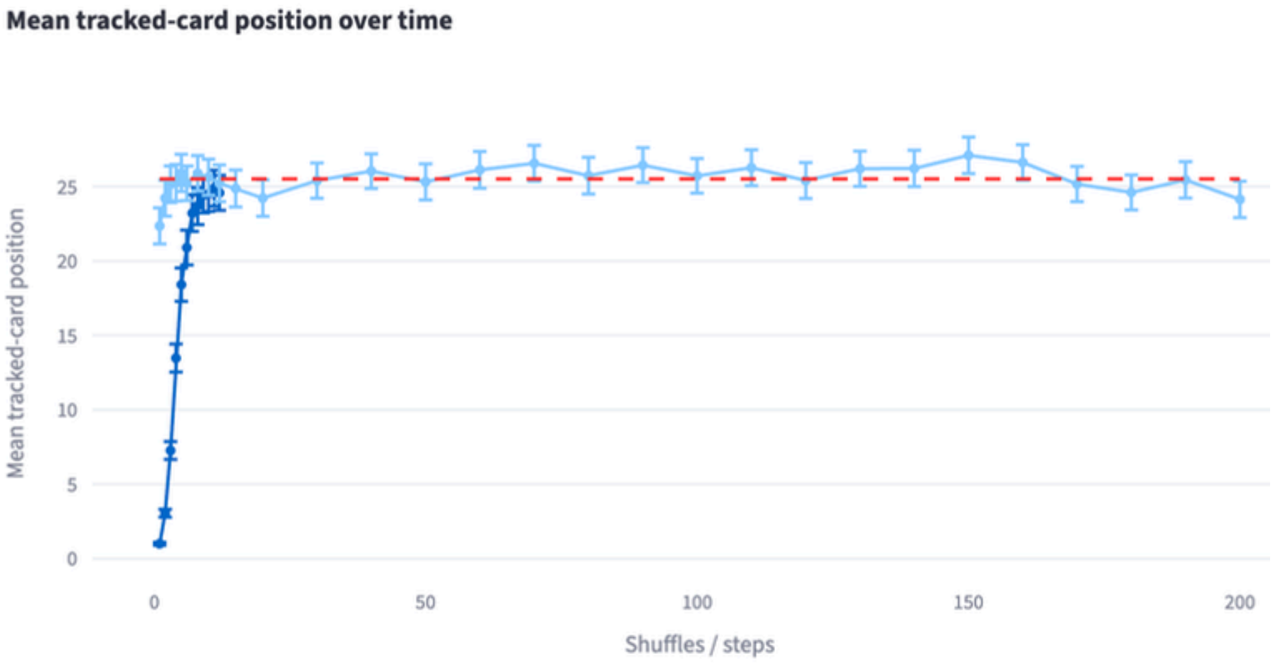


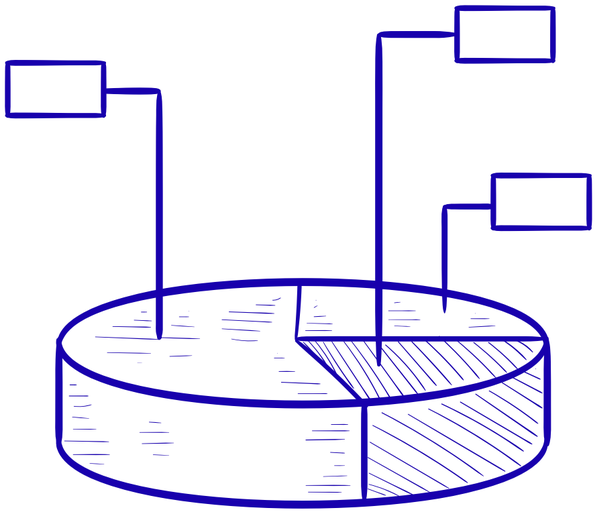



Measure how often order is broken locally, which answer “How fragmented is the deck into ordered chunks?”.

This counts how many increasing sequences (runs) appear in the deck, which measure how often order is broken locally

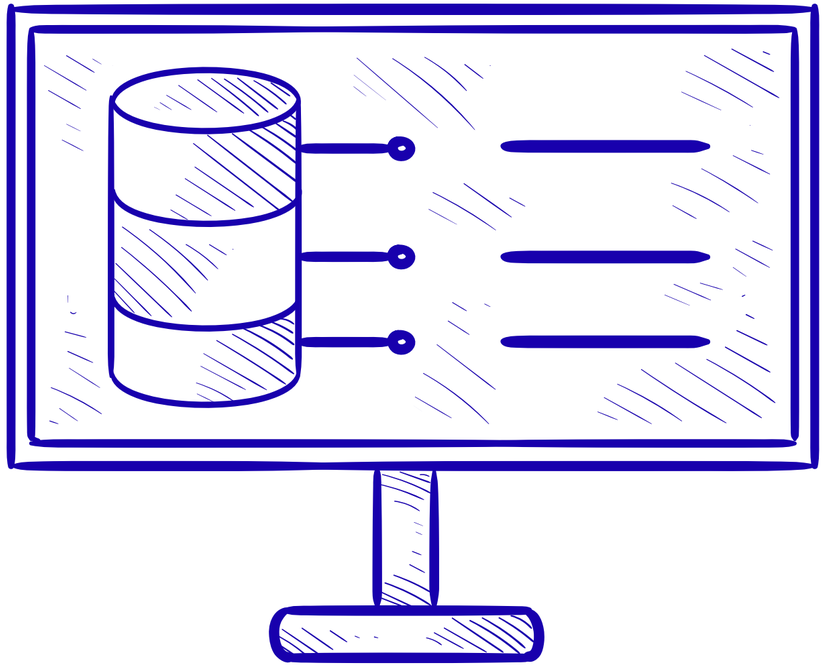
# MEAN POSITION



Summarize simulation results and quantify uncertainty, which answer “What is the typical value, and how stable is it?”.

This computes the average and variability of a metric across many trials. Typically show the mean position of the tracked card



# FEATURES

In this simulation, we achieve several features:

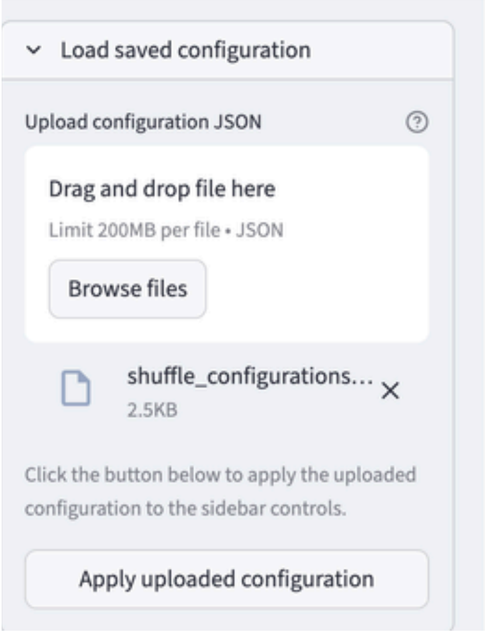
- Save / load shuffle configurations (JSON file)
- Highest & lowest probability histogram of the track card position distribution & 3D Visualization
- Track a specific card across positions
- Real-time metric visualization updates every N batches
- Multiple Metric Comparison & Configuration Comparison(overlay & side by side)
- Cached Computation

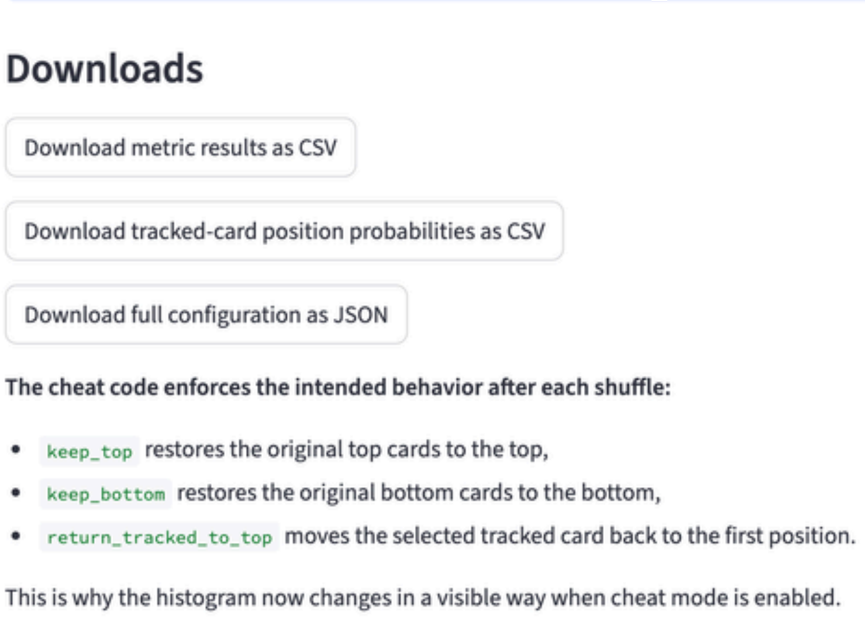
These features allow interactive exploration of randomness and make the simulation more than just static analysis.

# CONDIGURATION DATA STRUCTURE

The simulation have a page to download the configuration of the shuffle method.
We also get a window to upload the JSON file with the same structure

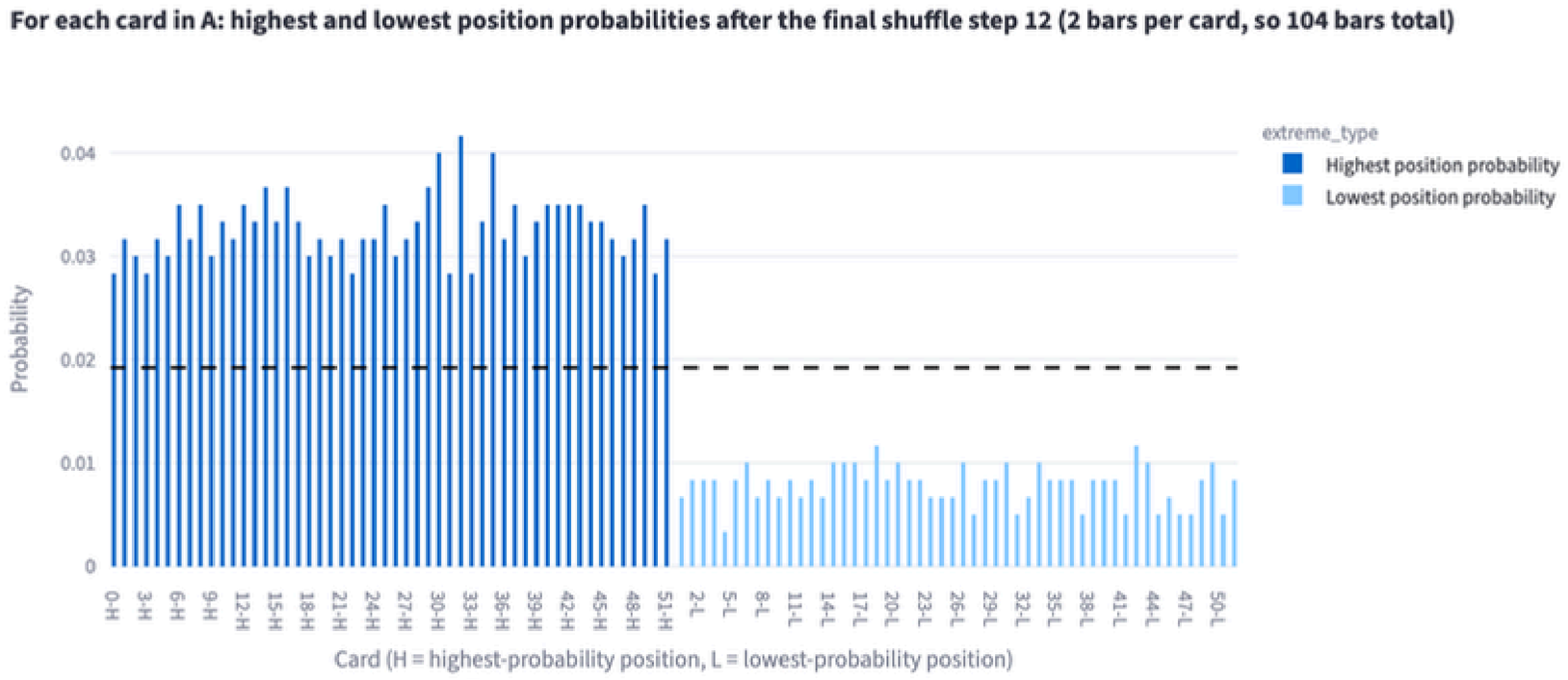

```json
{
  "app": "Card Shuffling Randomness Explorer",
  "version": 1,
  "common_settings": {
    "deck_size": 52,
    "tracked_card": 0,
    "trials": 600,
    "seed": 2026,
    "batch_size": 100,
    "max_workers": 2,
    "refresh_every": 2,
    "use_cached_final": false,
    "compare_enabled": true,
    "config_count": 2
  },
  "display_settings": {
    "metric_choice": "pos_mean",
    "page_mode": "Downloads",
    "chart_layout_mode": "Overlay",
    "metric_view_mode": "Single metric",
    "metric_overlay_choices": [
      "ks_pos",
      "pos_entropy_frac"
    ],
    "normalize_overlay_metrics": true,
    "metric_overlay_focus": "All visible configurations"
    "metric_compare_x": "ks_pos",
    "metric_compare_y": "pos_entropy_frac",
    "show_metric_compare_step_labels": true,
    "show_advanced_graphs": true,
    "show_runtime_table": true,
    "tracked_label_choice": "All visible configurations"
    "advanced_diag_label_choice": "A",
    "visible_labels": [
      "A",
      "B"
    ]
  },
  "shuffle_configurations": [
    {
      "prefix": "A",
      "display_name": "A",
      "method": "riffle",
      "schedule_settings": {
        "max_shuffles": 12
      },
      "steps_list": [
        1,
        2,
        3,
        4,
        5,
        6,
        7,
        8,
        9,
        10,
        11,
        12
      ],
      "shuffle_params": {
        "riffle_cut_p": 0.5,
        "p_overhand": 0.5,
        "piles_k": 7,
        "pile_random_pickup": true,
        "cheat_mode": "none",
        "cheat_cards": 2,
        "perfect_riffle": false,
        "perfect_riffle_start": "left"
      }
    },
```

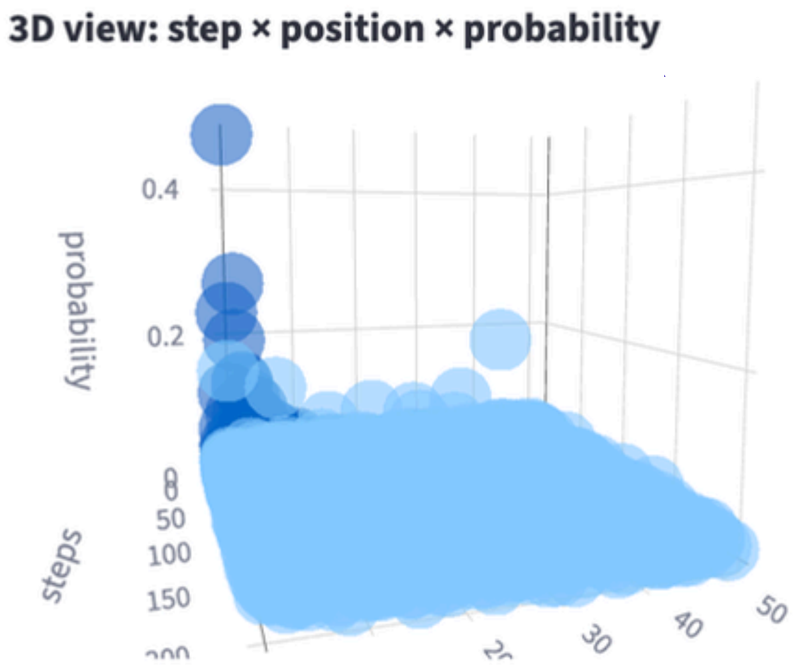



# HISTOGRAM & 3D VISUALIZATION

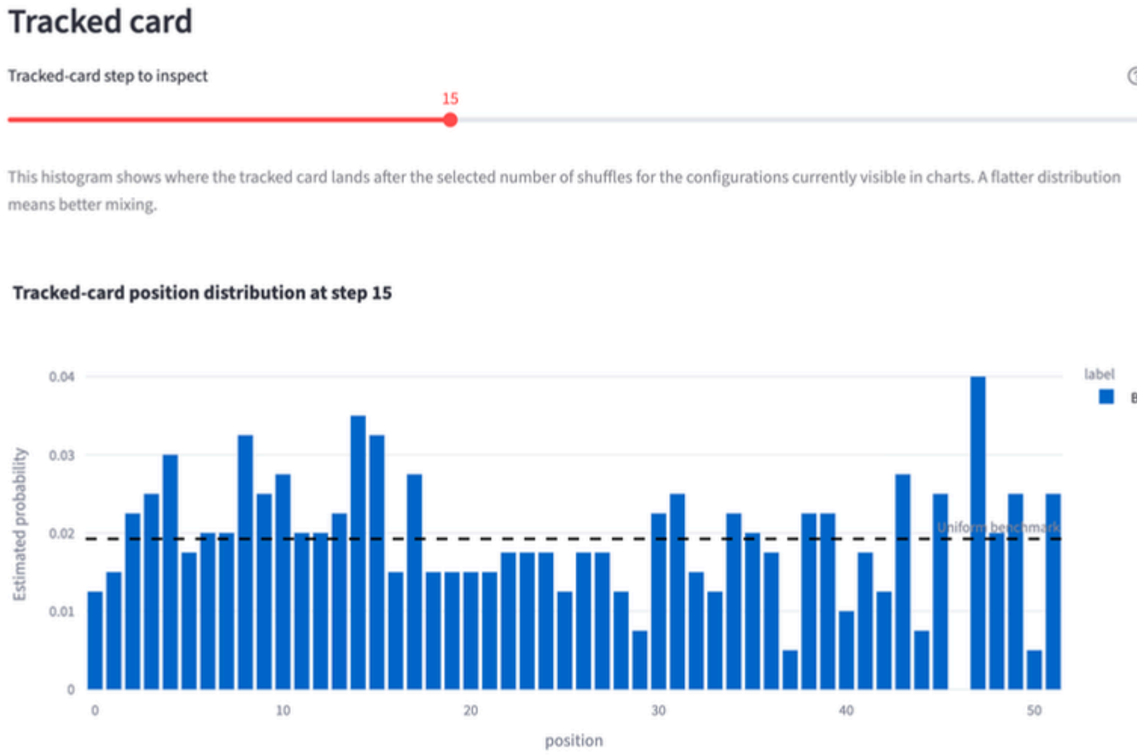
The highest/lowest chart always uses the selected configuration's final completed shuffle step.
The 3D view shows how the tracked-card probability mass moves across positions over shuffle steps for the visible configurations.

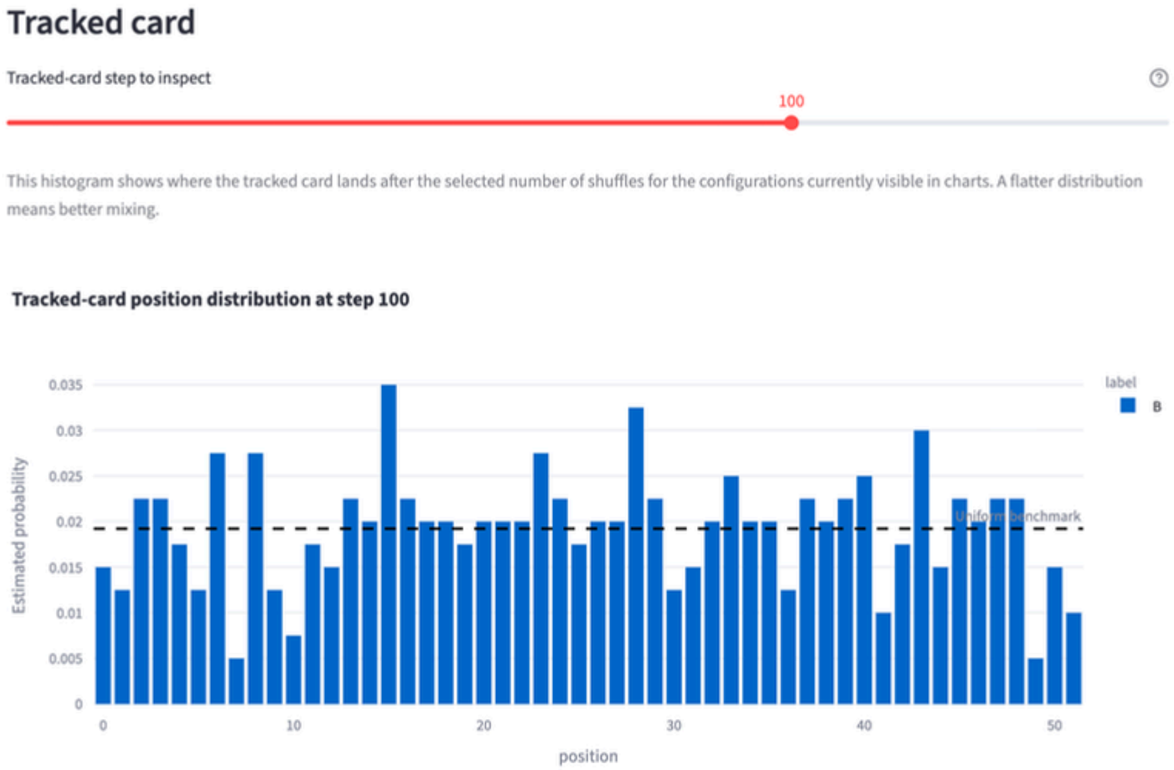


# TRACKED CARD POS DISTRIBUTION

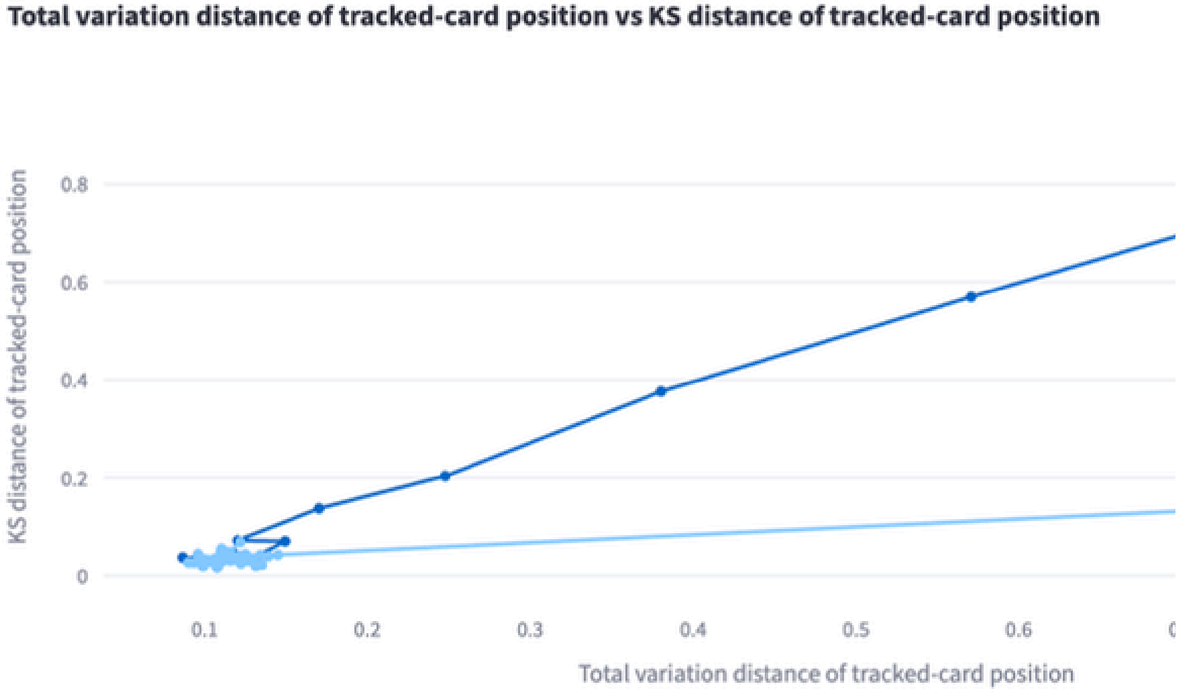
This histogram shows where the tracked card lands after the selected number of shuffles for the configurations currently visible in charts. A flatter distribution means better mixing.

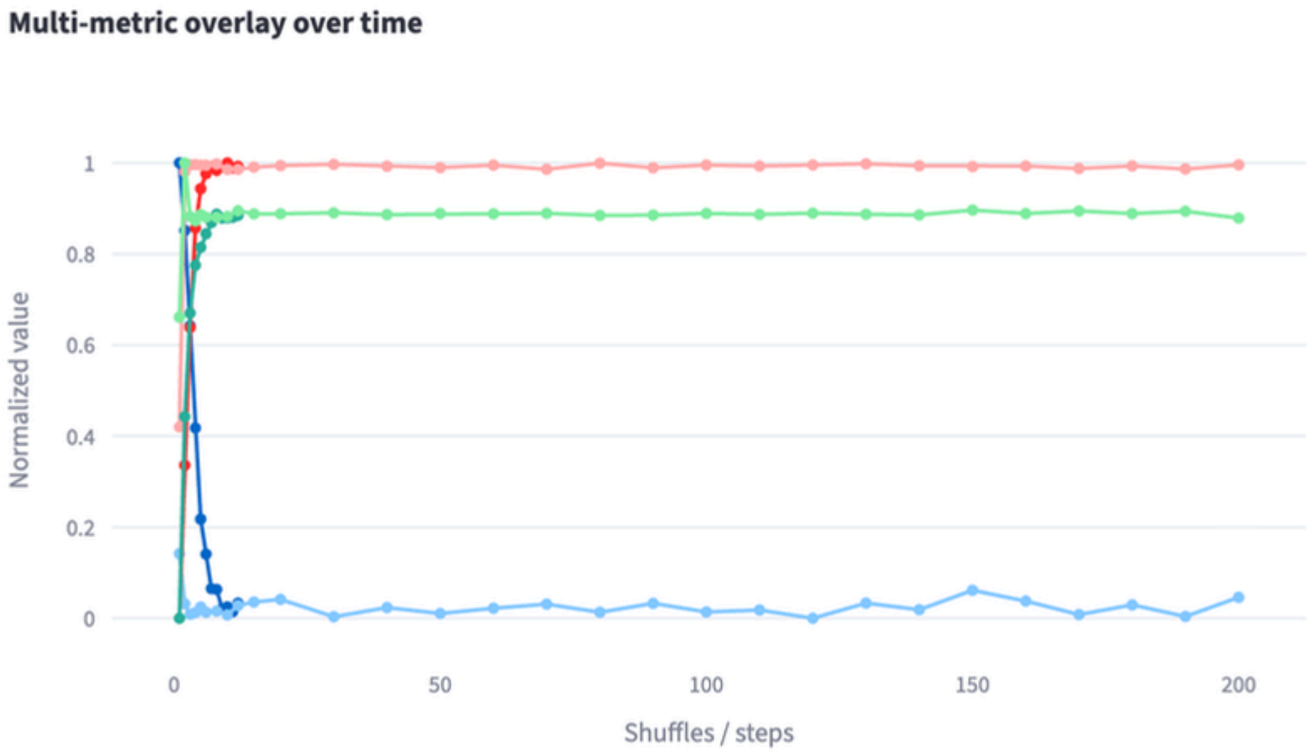


# METRIC COMPARISON & OVERLAY

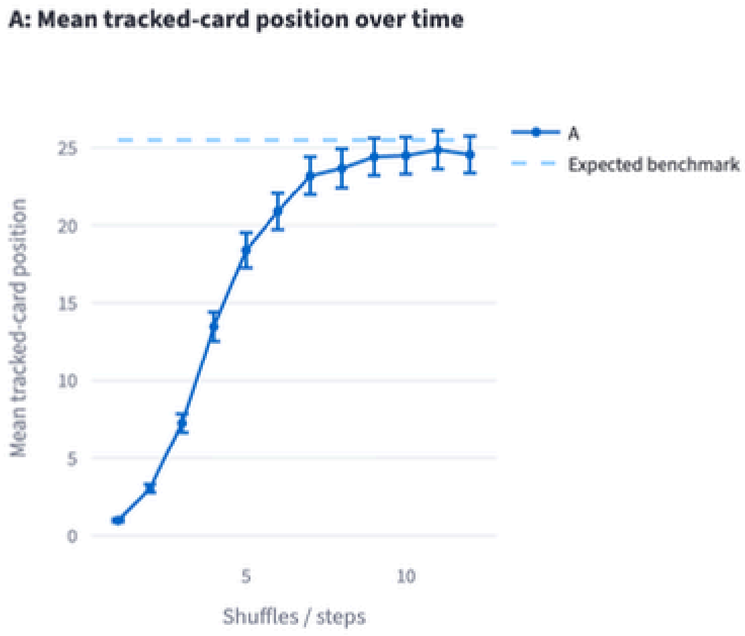
In this simulation, we realize the functionality that we can compare the metric side by side of different configurations or like shown below, do overlap comparison in the same diagram, either between different metric or between different shuffle

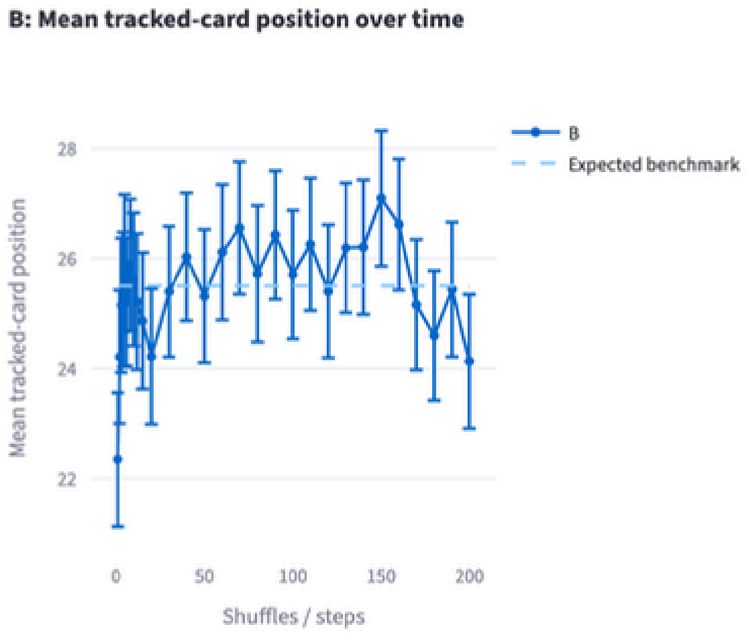


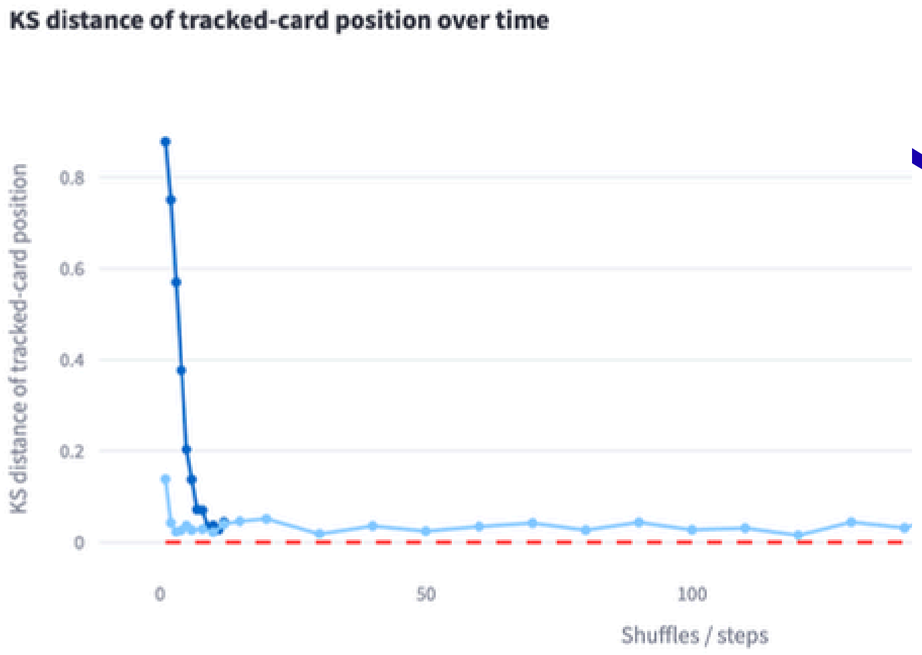



# CONFIGURATION COMPARISON & OVERLAY

In this simulation, we realize the functionality that we can compare the same metric of different shuffle configuration side by side, or, like shown below, do overlap comparison in the same diagram.



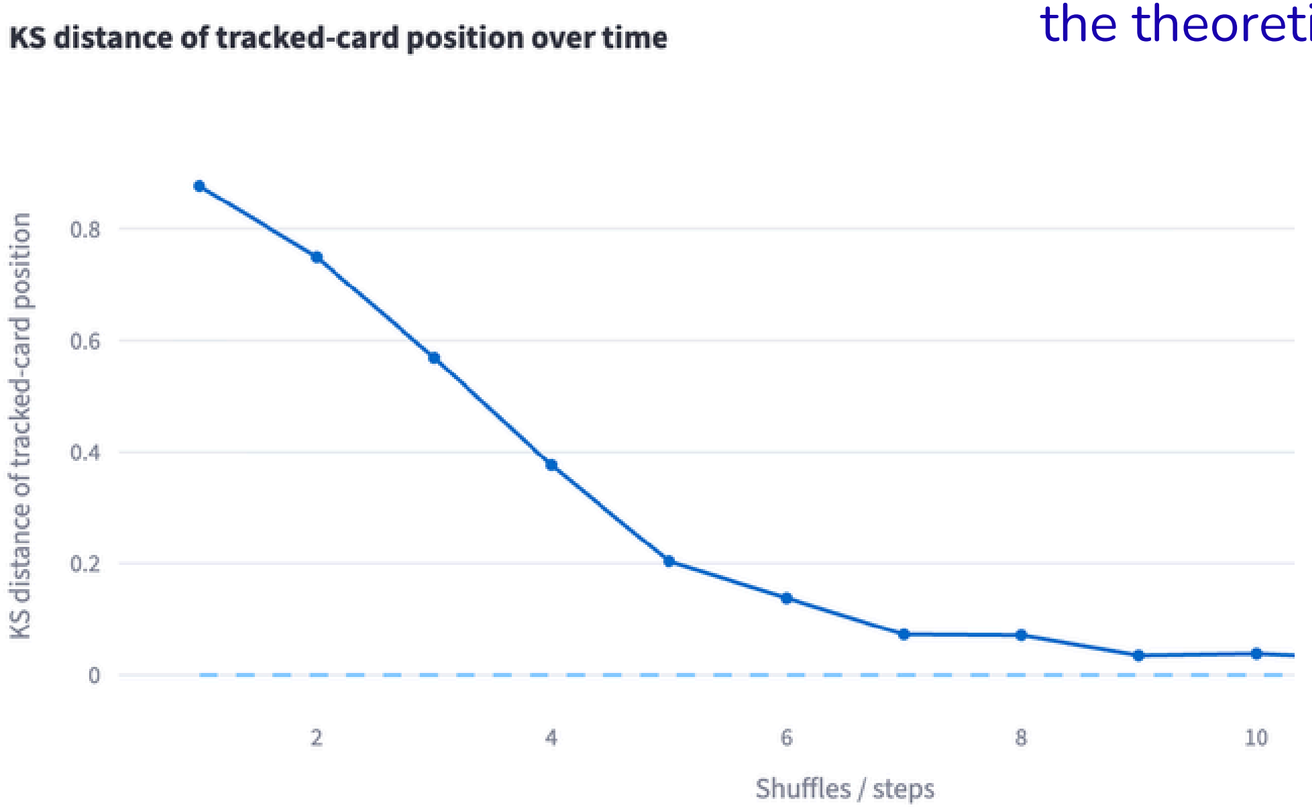



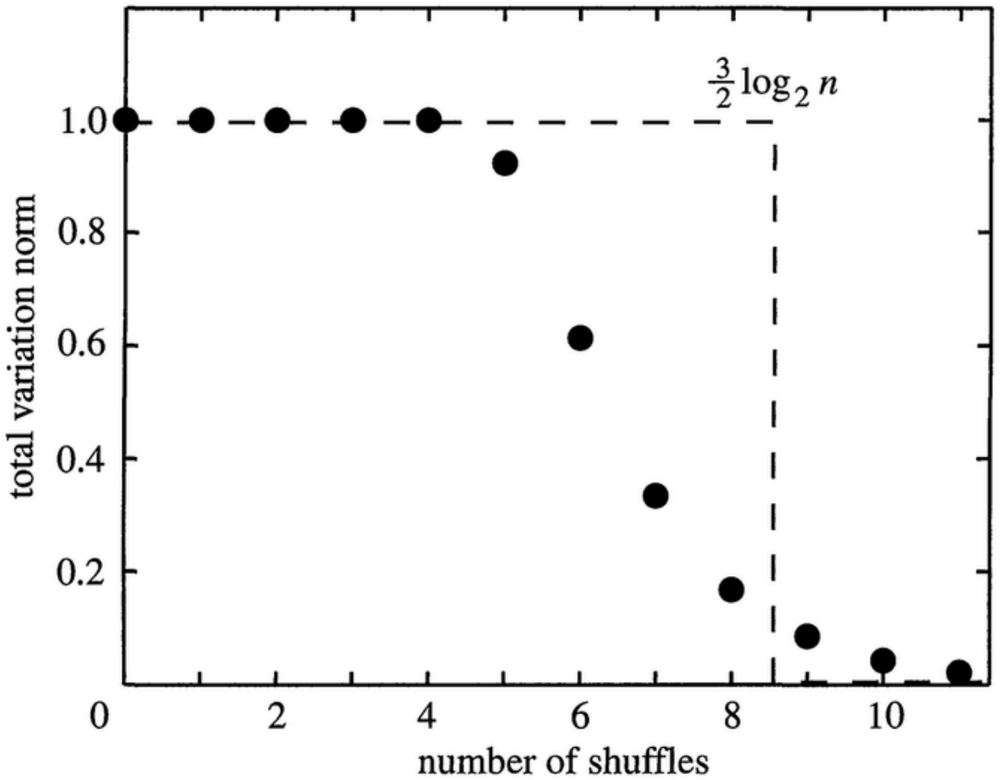



# Result
# RIffle

Compare our standard riffle shuffle KS distance with the graph from essay “How Many Shuffles to Randomize a Deck of Cards?”

We can clearly see that start from the $7_{th}$ iteration, both the KS distance and the TV distance of the entire deck approximate the benchmark, leaving just a small gap between the real value and the expected value for a perfect random card. This shows that our simulation successfully reproduces the theoretical result.



# Result Overhand

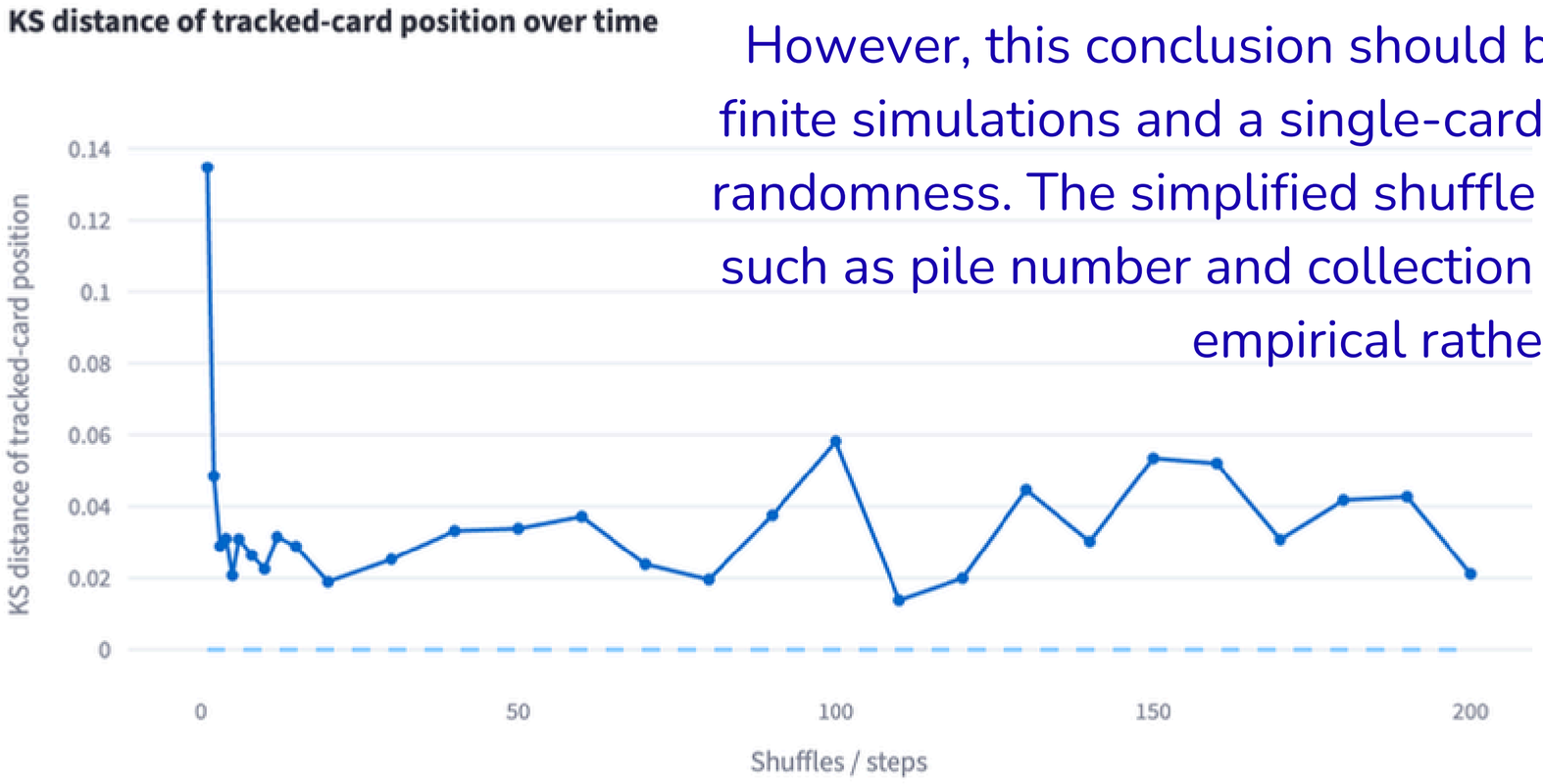
Although the KS distance drops sharply at the beginning, this only reflects the rapid destruction of the initial strong order in the deck. The shuffle quickly breaks obvious structure, leading to an early decrease in the metric. However, this does not mean the deck is already well mixed.

After this initial phase, the curve becomes much flatter, indicating that eliminating the remaining subtle biases takes significantly longer. This slow convergence dominates the overall mixing process, which is consistent with the theoretical result that the overhand shuffle has a mixing time on the order of $O(n^2 \log n)$.
to equilibrium.

This is typical in Markov chains: fast initial decay followed by slow convergence to equilibrium.

# Result Pile

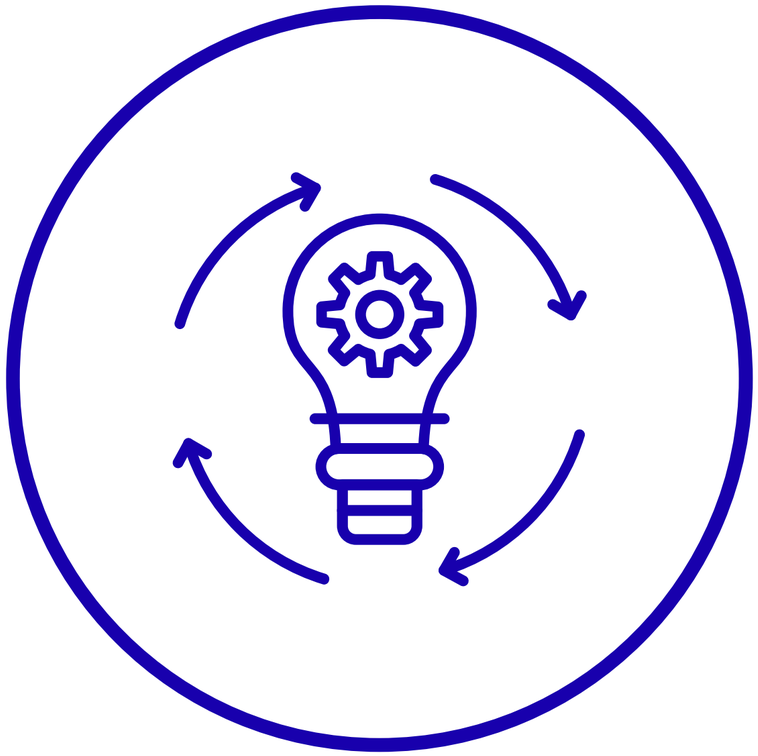
The pile shuffle shows weaker mixing than other methods. Although the KS distance drops initially as obvious structure is broken, it soon stabilizes and fluctuates around a small but non-zero level. This indicates that while some randomness is introduced, underlying bias remains even after many iterations. Unlike riffle shuffle, it does not clearly converge to uniformity, reflecting low mixing efficiency.




However, this conclusion should be treated with caution. It is based on finite simulations and a single-card metric, which may not capture global randomness. The simplified shuffle model may also vary with parameters such as pile number and collection order, so the result should be seen as empirical rather than theoretical.

# REFLECTION

The following shows the feedback and comment about this project

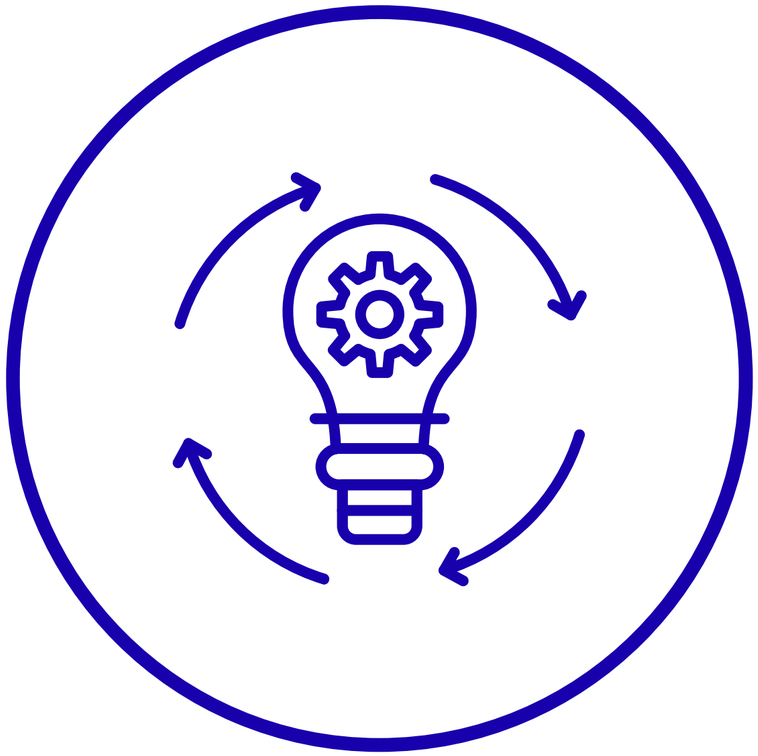


# EARLY MISTAKES IN MY PROJECT

At the beginning of this project, I went in the wrong direction both in theory and implementation.

On the theory side, I focused too much on finding “the correct answer” from papers, such as how many shuffles are needed, without fully understanding their assumptions. Later, I realized that different definitions of randomness lead to different answers, so I shifted to comparing them through simulation.

On the implementation side, I built everything too quickly without a clear structure. My code was not modular, so when I added new metrics or shuffle methods, I had to rewrite parts of it multiple times.

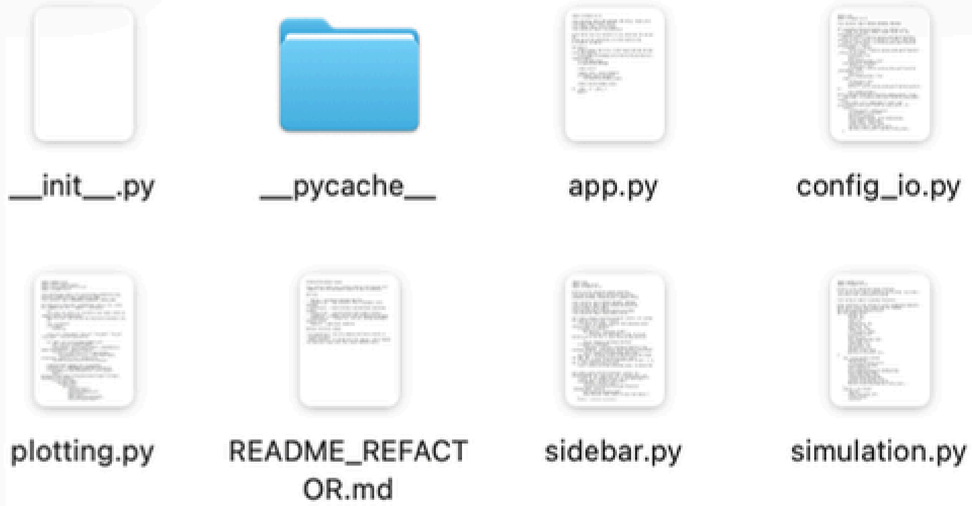


# USING GITHUB IN THIS PROJECT

During this project, I revisited the basic use of GitHub and version control. I practiced making commits, pushing changes, and managing different versions of my code as the project developed.

Through this process, I became more familiar with how to track changes and organize my work over time. For example, when I made multiple updates to the simulation or visualization parts, committing regularly helped me keep different stages of the project clear.

This experience helped me understand how version control supports the development process, especially in a project that evolves step by step.



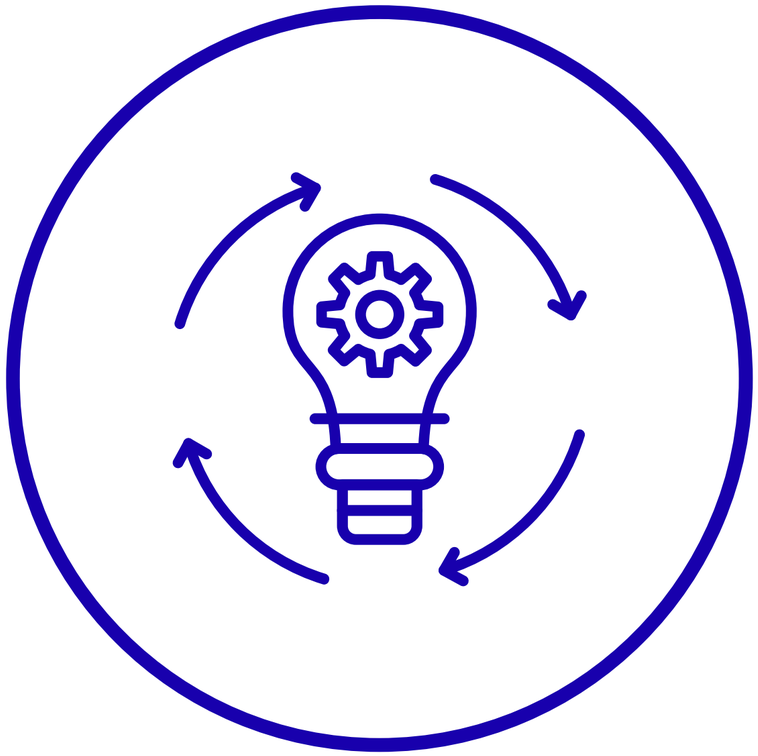

```python
    st.subheader("Overview")
    if not metrics_df.empty:...
    else:
        st.info("No metric data available for the selected visible configurations.")

elif page_mode == "Tracked card":
    st.subheader("Tracked card")
    tracked_label_choice = sidebar_state.get("tracked_label_choice", "All visible configurations")

    if tracked_label_choice == "All visible configurations":
        source_pos_df = pos_df.copy()
    elif tracked_label_choice == "All configurations":
        source_pos_df = all_pos_df.copy()
    else:
        source_pos_df = all_pos_df[all_pos_df["label"] == tracked_label_choice].copy()

    if not source_pos_df.empty:
        chosen_tracked_step, available_tracked_steps = choose_tracked_step(source_pos_df, tracked_label_choice)
    else:
        chosen_tracked_step, available_tracked_steps = None, []

    if not source_pos_df.empty and chosen_tracked_step is not None:
        tracked_view_df = source_pos_df.copy()

        if tracked_view_df.empty:
            st.info("The selected tracked-card configuration has no available data for the current run.")
        else:
```

# CODE COMMENTS

**GPT Reflection:**

The main issue with your current app.py is that it has become too heavy in terms of responsibilities. It handles page layout, Plotly chart construction, sidebar state, configuration I/O, and simulation orchestration all in one place. While this works early on, your growing number of chart functions and page modes already make the file harder to read and maintain.

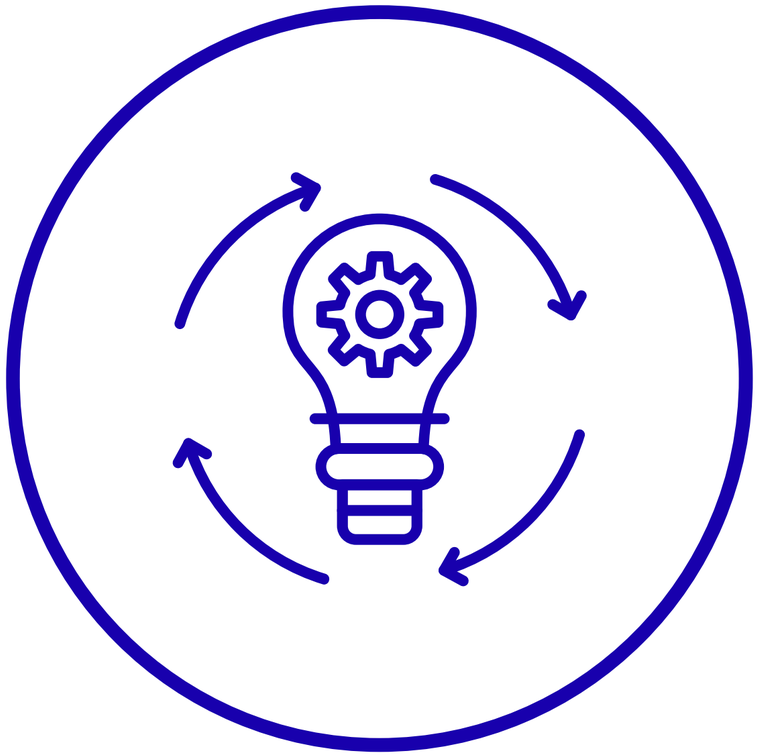
A better direction is to modularize the code: move plotting logic into plotting.py, sidebar and session state into sidebar.py, configuration handling into config_io.py, and simulation execution into simulation.py, while keeping app.py mainly for entry and routing. If you're not ready for a full refactor, a good first step is to extract each page mode into separate render functions to reduce complexity and prepare for cleaner modularization.



# DESIGN, CHALLENGES & KEY INSIGHTS

I designed the project in three layers: the mathematical layer (defining randomness and metrics), the simulation layer (running repeated shuffle experiments), and the interface layer (visualizing results using Streamlit). This structure helped connect theory with actual results.

One of the biggest challenges was choosing appropriate metrics. Some metrics are mathematically accurate but hard to interpret, while others are intuitive but less precise. To address this, I used multiple metrics to evaluate randomness from different perspectives.

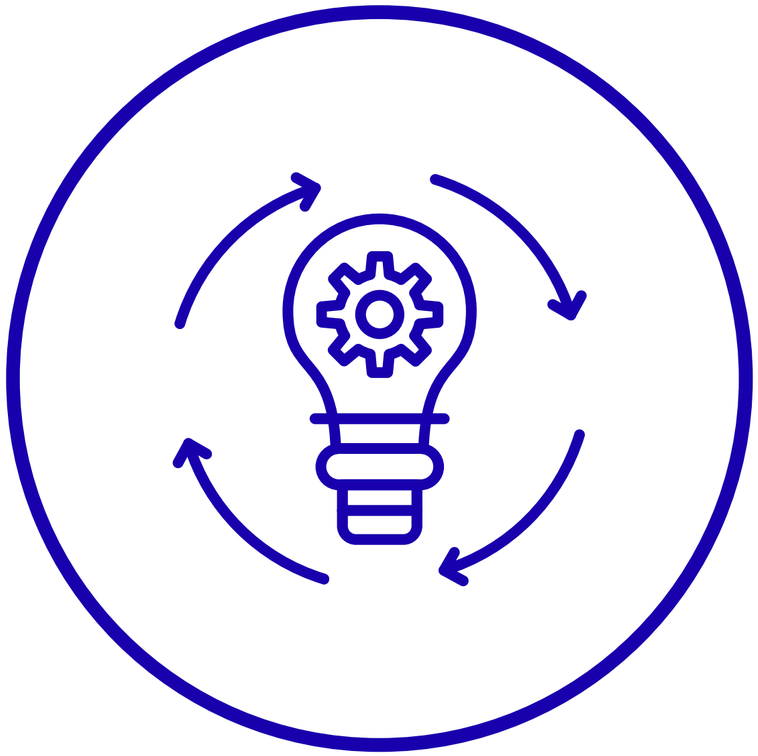


# WHY DEFINE FUNCTIONS NOT CLASSES

In this project, I mainly used a functional programming style, where each part of the system is implemented as independent functions. For example, shuffle methods and simulation logic are implemented separately and can be reused

This makes the code easier to test, allows it to stay close to the underlying mathematical definitions, and provides flexibility for experimentation.

However, as the project got bigger, I found that some parts of the code became harder to manage, especially when different components needed to work together. This made me realize that although using functions is simple at the beginning, a more structured design might make the project easier to extend and maintain in the long term.



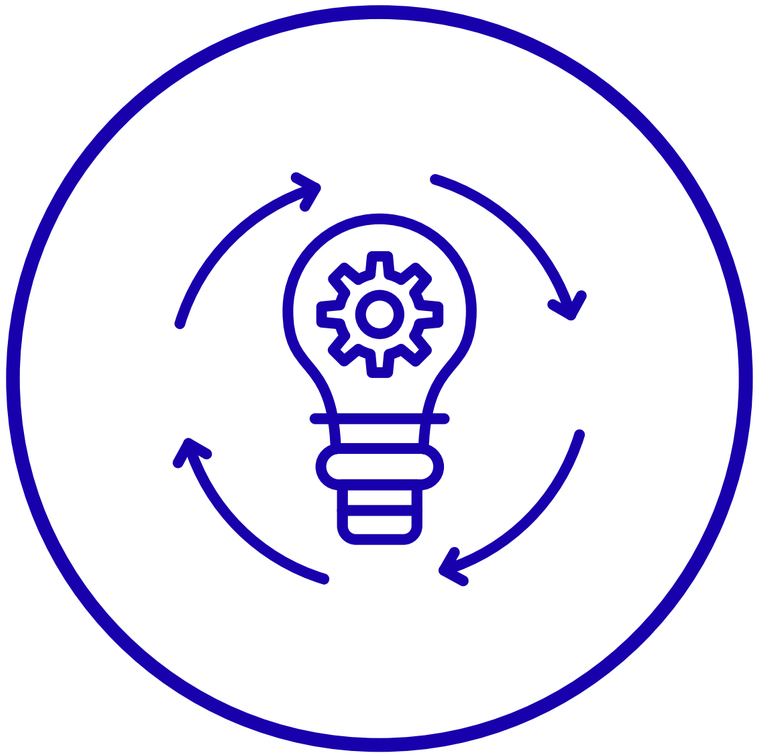
# HOW TO UPGRADE MY CODE

While developing the project, I realized that my code structure was not well planned at the beginning, which led to frequent modifications during development.

For example, the pipeline from simulation to visualization was not clearly separated at first. Functions for simulation, metric computation, and visualization were not clearly separated and often depended on each other., which made it harder to add new metrics or reuse results without modifying multiple parts of the code.

In future work, I would design this pipeline more clearly, such as separating configuration, simulation, data processing, and visualization into well-defined stages. I would also define consistent data formats for passing results between functions.

This would reduce repeated changes and make the system easier to extend and maintain.

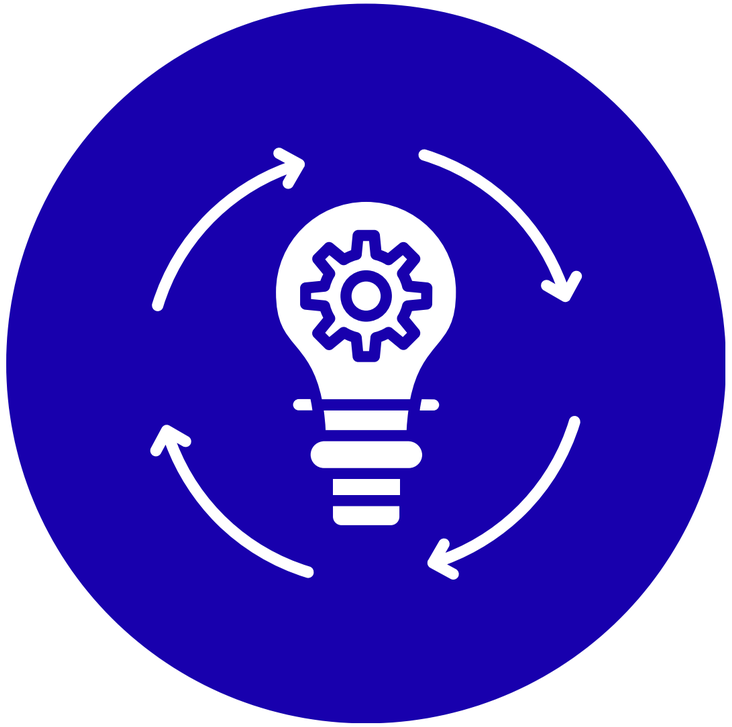


# STREAMLIT & VISUALIZATION

Using Streamlit allowed me to turn my simulation into an interactive tool instead of just printing numbers. I connected the backend functions (like run_trials) directly to the UI, so users can choose shuffle methods, adjust parameters, and immediately see updated results, which make the project more like a tool rather than a script.

I also used Plotly to create dynamic visualizations, including line charts for how metrics change over shuffles and bar charts for tracked-card distributions. These are generated from functions like make_metric_figure, so the graphs update automatically whenever the input changes.

Another thing I learned is how to design for user understanding, not just computation. For example, I added features like: tracking a single card; showing benchmark lines; comparing multiple shuffle methods in one chart,... etc
These design choices helped make abstract concepts like randomness much easier to understand visually.

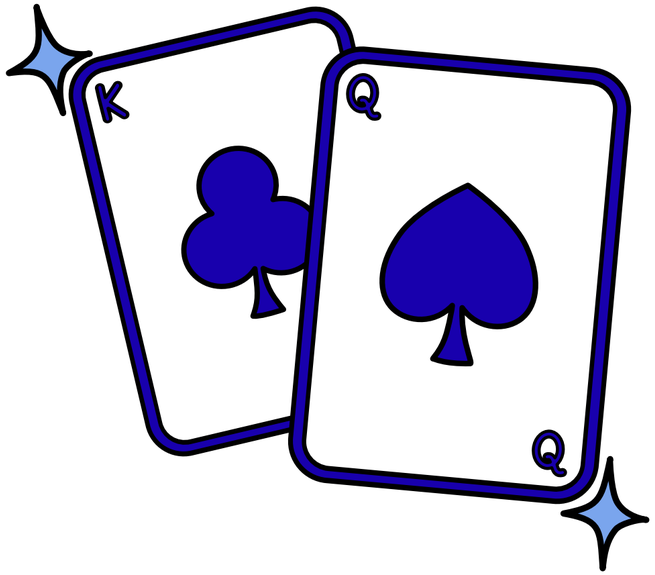


# WHAT I LEARNED FROM THIS PROJECT

By writing the code myself, I learned how to turn a mathematical idea into an actual algorithm.For example, in my riffle shuffle, I didn't just randomly reorder the deck. I simulated a binomial cut and then merged two piles based on probabilities of remaining cards. This forced me to think carefully about how each card moves step by step, which made the theory much clearer.

I also learned how to build a full simulation system instead of just separate functions.
In my project, I used a configuration system (build_shuffle_config) to define different shuffle methods, then ran many trials using run_trials, and finally converted results into data for visualization. This helped me understand that randomness cannot be judged from a single shuffle — it needs repeated simulations and aggregated results.

Finally, I learned how to organize and optimize code in a real project. I separated shuffle logic, simulation, an visualization so they can work together, and I used tools like parallel execution (ThreadPoolExecutor) and caching (@st.cache_data) to improve performance. This made me realize that coding is not just about correctness, but also about structure, efficiency, and making the system usable.




# Thank You